

Tribhuvan University
Final Year Project Report — BSc CSIT / BE

Nepal FLDS

An AI-Powered Flood & Landslide Detection and Early Warning System for
Nepal

Machine Learning · XGBoost

Node.js · Express · MongoDB

Python · FastAPI

Real-Time Weather (Open-Meteo)

Project: Flood & Landslide Detection System (Nepal FLDS)

Domain: Disaster Risk Reduction & Early Warning

Coverage: All 77 Districts of Nepal

Model Version: XGBoost v1.2.0

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement & Objectives
4. Literature Review & Related Work
5. System Architecture
6. Machine Learning Model Architecture
7. Data Pipeline & Live Weather Ingestion
8. Backend Architecture (Node.js / Express / MongoDB)
9. Frontend Architecture
10. Cross-Verification & Ground-Truth Confirmation
11. Security & Reliability
12. Testing & Evaluation
13. Deployment & Operations
14. Limitations & Future Work
15. Conclusion

1. Abstract

Nepal FLDS is a full-stack, AI-powered early warning system that continuously monitors weather conditions across all 77 districts of Nepal and predicts the risk of floods and landslides in near real time. Live weather data is fetched every few minutes from the Open-Meteo API, processed through a data pipeline, and scored by two Gradient Boosted Decision Tree (XGBoost) classifiers — one for flood and one for landslide risk. When the model judges a district to be at moderate or higher risk, an alert is created and pushed instantly to every connected dashboard over a Server-Sent Events (SSE) feed. Every alert is independently cross-checked against the documented monsoon rainfall trigger thresholds the models were trained on, and responders can record field confirmations, producing an honest measure of "model confidence vs confirmed events." The system was trained on 241,318 district-days of real observed weather per model (2018–2026) and achieves 99.81% flood and 99.74% landslide classification accuracy on held-out data.

Key Metrics

77

DISTRICTS COVERED

241,318

TRAINING ROWS /
MODEL

99.81%

FLOOD ACCURACY

99.74%

LANDSLIDE ACCURACY

~10 min

FAST-TRACK RE-EVALUATION

2. Introduction

Nepal is one of the most disaster-prone countries in the world. Its steep Himalayan topography, young and fragile geology, and intense monsoon rainfall combine to make floods and landslides recurring, life-threatening hazards. Between 2010 and 2021, floods and landslides in Nepal killed thousands of people and caused billions of rupees in damage; events such as the 2021 Melamchi flood demonstrated how quickly a single rainstorm can devastate entire valleys.

The most effective mitigation measure for weather-driven hazards is **early warning**. If a district can be told hours in advance that rainfall conditions are approaching dangerous thresholds, communities can move to higher ground, traffic can be diverted, and emergency teams can preposition. However, traditional warning systems rely on sparse physical sensor networks (river gauges, rain gauges) that are expensive to install and maintain across 77 mountainous districts.

This project addresses that gap with a different approach: instead of requiring a physical sensor in every district, it uses **freely available satellite- and station-derived weather data** (Open-Meteo) as the sensor layer. Machine learning models learn the historical relationship between observed weather — daily rainfall, its accumulation over 3 and 7 days, humidity, temperature, wind, soil moisture — and the documented monsoon rainfall thresholds that precede floods and landslides. The result is a system that can reason about every district in Nepal with no hardware deployment at all, and that surfaces its predictions on a modern, real-time web dashboard built for disaster-management operators.

2.1 Motivation

- Nepal's rugged terrain makes physical sensor coverage incomplete and uneven.
- Monsoon flooding and landslides are driven by rainfall patterns that are now forecastable in near real time.
- Free weather APIs make a nationwide, sensor-free early warning system technically and financially feasible.
- An honest early warning system must distinguish *predicted risk* from *confirmed events* — this project explicitly models both.

2.2 Scope

The system covers flood and landslide risk for all 77 districts. Earthquake prediction is intentionally excluded — the physical processes behind earthquakes are not predictable from weather, and fabricating quake predictions would undermine the system's credibility. The project scope includes:

- A **machine learning training pipeline** (Python) that downloads historical weather, derives risk labels from documented thresholds, trains XGBoost classifiers, and records full provenance.
- A **prediction microservice** (Python FastAPI) that serves the trained models.
- A **backend API and orchestration layer** (Node.js, Express, MongoDB) that polls live weather, runs predictions, raises alerts, and streams them in real time.
- A **responsive web frontend** (HTML/CSS/JS, Chart.js, Leaflet) with dashboard, alerts, map, predictions, and reports pages.

3. Problem Statement & Objectives

3.1 Problem Statement

Floods and landslides during Nepal's monsoon season regularly cause loss of life and property. Existing warning mechanisms are limited by sparse physical sensor infrastructure, high deployment cost, and slow dissemination of warnings. There is a need for a low-cost, nationwide, near-real-time early warning system that (a) continuously monitors all districts, (b) converts live weather into an actionable risk assessment using data-driven models, (c) distributes alerts instantly to operators, and (d) honestly tracks whether predicted risks were confirmed on the ground.

3.2 Objectives

1. Design and train machine learning models that classify flood and landslide risk from historical weather data for all 77 districts of Nepal.
2. Build a live data pipeline that ingests real weather observations and feeds them through the trained models on a continuous schedule.
3. Implement a secure backend API with user authentication, role-based access, alert management, and real-time event streaming.
4. Develop an interactive frontend dashboard for monitoring alerts, viewing a risk map, running manual predictions, and reviewing reports.
5. Implement an independent cross-verification layer (rainfall-rule check) and a ground-truth confirmation workflow so the system reports honest, explainable numbers.
6. Deploy the system with automated service supervision so it runs continuously with minimal operator intervention.

4. Literature Review & Related Work

4.1 Machine Learning for Flood & Landslide Prediction

Gradient Boosted Decision Trees, particularly XGBoost (Chen & Guestrin, 2016), are a standard, highly effective choice for tabular hazard prediction. XGBoost builds an ensemble of shallow decision trees where each new tree corrects the residual errors of its predecessors. It handles mixed feature types, missing values, and non-linear interactions well, and it routinely outperforms deep neural networks on small-to-medium tabular datasets — exactly the regime of this project (a few hundred thousand rows, ~12 features).

Studies of flood forecasting in the Hindu Kush–Himalaya region consistently identify **accumulated rainfall over 3–7 days** as the dominant trigger for both river floods and shallow landslides, alongside soil saturation and local terrain. Nepal's Department of Hydrology and Meteorology (DHM) and the National Disaster Risk Reduction and Management Authority (NDRRMA) publish monsoon rainfall thresholds used operationally by forecasters. This project encodes those same documented thresholds as the training-label rule (Section 6.4) and reuses them at runtime as an independent verification rule (Section 10), which is a novel, defensible design choice for a student project.

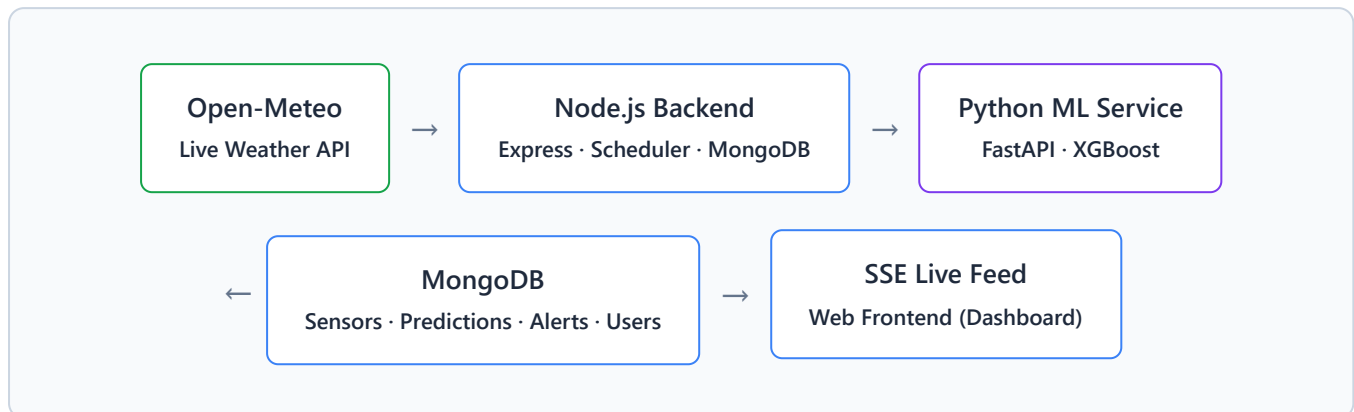
4.2 Related Systems

System	Approach	Coverage	Limitation Addressed Here
DHM / NDRRMA bulletins	Manual, forecaster-led	Selected basins	Not automated, not nationwide, not real-time to citizens
GLOFAS (Global Flood Awareness System)	Global hydrological model	Global rivers	Coarse resolution; not tuned to Nepal's districts
IoT river-gauge systems	Physical sensors + thresholds	Few sites	High cost; sparse coverage
Nepal FLDS (this project)	XGBoost on free weather data	All 77 districts	Nationwide, sensor-free, real-time, self-verifying

Honest framing: this project predicts *risk from weather*. It does not claim to replace authoritative hydrological models. Its contribution is the end-to-end engineering: nationwide automated risk scoring, real-time dissemination, and an explicit, auditable gap between predicted risk and confirmed events.

5. System Architecture

The system is a three-tier application with an additional ML tier and a live-data source:



5.1 Components

Tier	Technology	Responsibility
Data Source	Open-Meteo API (free, no key)	Live weather: temperature, humidity, precipitation, wind, soil moisture; plus 7-day historical window for rain accumulation
Backend	Node.js, Express, Mongoose, node-cron	REST API, authentication, scheduling, alert lifecycle, SSE streaming, static frontend serving
ML Service	Python, FastAPI, XGBoost, joblib	Loads trained pickles, builds feature vectors, returns risk class + confidence
Database	MongoDB	Users, sensor readings, predictions, alerts (with GeoJSON + verification metadata)
Frontend	HTML/CSS/JS, Chart.js, Leaflet	Dashboard, alerts, map, predictions, reports; live updates via SSE

5.2 Request Flow (Live Poll Cycle)

- `predictionScheduler` fires every `EXTERNAL_POLL_INTERVAL` minutes (default 10).
- `externalDataService` fetches live weather for each of the 77 districts (concurrency-limited to avoid rate limits).
- Each reading is stored as a sensor document, then sent to the ML service via `POST /predict` for flood and landslide.
- The backend stores the prediction; if risk class $\geq$ moderate, it creates an alert, notifies users, and emits `alert:new` over SSE.
- The frontend receives the SSE event and updates the dashboard instantly without a page reload.

6. Machine Learning Model Architecture

6.1 Model Choice: XGBoost

XGBoost (eXtreme Gradient Boosting) was chosen over deep neural networks and other classical classifiers for four reasons grounded in this problem's shape:

- **Tabular data:** the features are ~12 heterogeneous numeric quantities; tree ensembles dominate on such inputs without the need for feature scaling or embedding layers.
- **Interpretability:** feature importance and per-sample SHAP-style reasoning are straightforward, supporting the project's explainability requirement.
- **Robustness:** built-in handling of missing values and resistance to outliers matter when the data source is a public weather API.
- **Efficiency:** fast training and inference; a 300-tree model scores a district in milliseconds, comfortably supporting 77-district live polling.

6.2 Hyperparameters

Parameter	Flood Model	Landslide Model
Algorithm	XGBoost (gbtree)	XGBoost (gbtree)
Number of trees	300	300
Max depth	6	6
Objective	multi:softprob (3 classes)	multi:softprob (3 classes)
Training rows	241,318	241,318
Classes	0 = low, 1 = moderate, 2 = high	0 = low, 1 = moderate, 2 = high

6.3 Features

Flood Model (12 features)	Landslide Model (11 features)
temperature, rainfall, humidity, wind_speed, discharge, rainfall_roll3, rainfall_roll7, discharge_roll3, discharge_roll7, month, location_encoded, terrain_encoded	temperature, rainfall, humidity, wind_speed, rain_3day, rain_7day, soil_moisture, slope, month, terrain_encoded, location_encoded

- **Rolling rainfall (3/7-day)** encodes cumulative wetness — the single most important landslide and flood trigger.
- **location_encoded / terrain_encoded** are categorical encodings of district and terrain type (Hilly, Mountain, Terai), letting the model learn that the same rain is more dangerous on a slope than on flat Terai land.
- **month** captures seasonal context.

6.4 Training Labels — the "Monsoon Rule"

Because no free historical dataset of *confirmed* flood/landslide events exists with full district coverage, labels are derived from observed rainfall using the documented monsoon trigger thresholds. This is a defensible, reproducible labeling strategy: the model learns "given this weather, how likely is the situation to meet the documented danger thresholds."

Disaster	Moderate (class 1)	High (class 2)
Flood	≥ 30 mm/day OR ≥ 60 mm/7d	≥ 80 mm/day OR ≥ 150 mm/7d
Landslide	≥ 20 mm/day OR ≥ 40 mm/3d	≥ 60 mm/day OR ≥ 100 mm/3d

6.5 Class Balance & Evaluation

Class	Flood samples	Landslide samples	Flood F1	Landslide F1
0 (low)	186,851	206,656	0.9995	0.9994
1 (moderate)	39,011	27,343	0.9940	0.9887
2 (high)	15,456	7,319	0.9912	0.9742
Weighted avg	Flood acc 99.81%		Landslide acc 99.74%	

The "high" class is the rarest (flood 6.4%, landslide 3.0% of samples), yet still achieves 99%+ precision on held-out data. Precision on the high class (99.6% flood, 98.2% landslide) is the metric that answers the operational question "when it says *high*, how often is it right?"

6.6 Probability Calibration

XGBoost's class-confidence output is often near 1.0 for the winning class. Displaying raw confidence would make a moderate-risk alert read as "100% chance of disaster," which is misleading. The system therefore **caps the displayed probability to its risk band**: low $\leq 35\%$, moderate $\leq 60\%$, high $\leq 80\%$, critical $\leq 95\%$. The number always agrees with the label — a moderate alert can never display as near-certain.

7. Data Pipeline & Live Weather Ingestion

7.1 Training Data

Training uses the Open-Meteo **Archive API** (2018-01-01 → 2026-07-31), giving 8.5 years of daily observed weather per district. Each district-day becomes one training row; labels come from the monsoon rule (Section 6.4). Districts that fail to download (quota/network) are automatically retried with backoff until covered — the metadata records 77/77 districts trained, 0 pending.

7.2 Live Data (Inference)

Live inference uses the Open-Meteo **Forecast API** with `past_days=7`, so the same request returns today's conditions *and* the real accumulated 3/7-day rainfall needed for the rolling features. The per-district polling is concurrency-limited (8 workers) and windowed to respect the free tier:

Window	Value	Purpose
EXTERNAL_POLL_INTERVAL	10 min	Scheduler cadence
EXTERNAL_POLL_WINDOW_MIN	30 min	Calm districts — re-evaluation window
EXTERNAL_FAST_TRACK_WINDOW_MIN	10 min	Districts with an active moderate+ alert — re-checked 3× more often

The **fast-track** mechanism is the key real-time optimization: resources concentrate on districts already in danger while calm districts poll at the budget-friendly rate. Deduplication keys (bucket timestamp per district per window) ensure each district+type is evaluated exactly once per window.

7.3 Reading → Prediction → Alert



8. Backend Architecture (Node.js / Express / MongoDB)

8.1 Project Structure

```
backend/  
  src/  
    config/      constants (risk levels, caps, rule thresholds), db connection  
    controllers/ auth, prediction, alert, sensor, dashboard  
    jobs/        predictionScheduler (cron: sensor, cleanup, external poll)  
    middleware/  auth (JWT + API key), ratelimiter, errorHandler  
    ml/          modelBridge (Python ML service client + local TF.js fallback)  
    models/      User, SensorData, Prediction, Alert (Mongoose schemas)  
    routes/      auth, predictions, alerts, sensors, dashboard, events (SSE)  
    services/    alertService, predictionService, externalDataService,  
                notificationService, liveEventBus  
    utils/       dataParser, geoHelper, logger  
    scripts/     seed-demo, poll-external, import-csv, cleanup-dummy-earthquakes  
    tests/       app.test.js (Jest + Supertest)
```

8.2 REST API

Area	Endpoints
Auth	POST /auth/register · POST /auth/login · GET /auth/me · PUT /auth/profile · PUT /auth/change-password
Predictions	POST /predictions · GET /predictions · GET /predictions/stats · GET /predictions/models/info · GET /predictions/:id
Alerts	GET /alerts · GET /alerts/stats · GET /alerts/:id · PATCH /alerts/:id/resolve · PATCH /alerts/:id/confirm
Sensors	POST /sensors/data · GET /sensors/data · GET /sensors/latest
Dashboard	GET /dashboard/overview
Live	GET /events/stream (SSE)

8.3 Data Models

- **User** — name, email, bcrypt-hashed password, role (user/admin/responder), GeoJSON location, alert preferences (email/SMS, disaster types, min risk level).
- **SensorData** — sensor id/type, disaster type, GeoJSON location, readings object (all hazard fields), raw payload, prediction linkage.
- **Prediction** — disaster type, calibrated probability, risk level, location, input data (the exact readings), model version, source (manual/sensor/external/csv-import/scheduled), verification object, alert linkage.
- **Alert** — prediction linkage, risk, probability, message, affected radius, notifications summary, verification copy, ground-truth confirmation (status/by/at/note), active + resolved timestamps.

8.4 Real-Time Streaming (SSE)

Server-Sent Events provide one-way push from the backend to every open dashboard. An in-process event bus (`liveEventBus`) lets services emit events without coupling: `alert:new`, `alert:resolved`, `alert:confirmed`, `prediction:new`, `sensor:ingest`. The SSE route authenticates via a token query parameter

(EventSource cannot set headers), forwards every event to subscribed clients, and sends 15-second heartbeats so proxies keep the connection alive. This gives true real-time updates with zero polling from the browser.

8.5 Notifications

Alert dispatch attempts email (Nodemailer) and SMS (Twilio) delivery to users within the alert's affected radius (geo query via the 2dsphere index). Both channels are *optional*: if credentials are unset, the system records `skipped` statuses and continues — the in-app/SSE path always works. Browser push notifications are also available on the frontend for high/critical alerts.

9. Frontend Architecture

9.1 Pages

Page	Contents
Dashboard (index.html)	8 stat cards (active alerts, critical zones, accuracy, rainfall peak, soil saturation...), weekly trend / 24h risk / radar charts, active alerts list with live-evidence chips, recent-events table, Leaflet map with alert markers, live activity timeline, SSE status
Active Alerts	Severity stat cards, filterable alert cards with evidence + verification badges, ground-truth summary strip, Details modal (live evidence, model version, confirm button), resolve + confirm actions
Nepal Map	Full-screen Leaflet map with per-district alert markers, event-type + province filter chips, live stats bar (flood/landslide/critical/high counts, districts alerted)
Predictions	Prediction history cards, model-info card (version, training data, per-model accuracy), province risk table, "Run Prediction" modal
Reports	Summary stats, monthly incident trend chart, incident breakdown donut, model-confidence-vs-confirmed card, recent reports table with filters

9.2 Frontend Stack

- **Vanilla JS modules** (`api.js` , `auth.js` , `live-data.js` , `live-feed.js` , `charts.js` , `map.js` , `interactions.js`) — no framework, no build step, served statically by the backend.
- **Chart.js** for trend/risk/donut/radar charts.
- **Leaflet + OpenStreetMap/CARTO tiles** for the district map.
- **Server-Sent Events** for live updates — new alerts, resolutions, and confirmations appear without refresh.
- **Dark dashboard theme** with custom CSS variables, responsive grid, and micro-interactions (toasts, modals, live status pill).

9.3 Data Display Rules (Honesty by Construction)

- All numbers come from the backend — there is **no mock fallback**; if the backend is offline a banner is shown and retries run automatically.
- Probabilities are capped to risk bands (Section 6.6) so a label and its number always agree.
- Alert cards show "live evidence" (the exact rainfall/humidity/temperature that drove the model).
- "✓ verified" badges appear only when the model and the rainfall rule agree; "confirmed/not-confirmed" badges come only from field reports.

10. Cross-Verification & Ground-Truth Confirmation

A disaster early-warning system must be auditable. This project implements two independent layers that make the system's output explainable and honest:

10.1 Rainfall-Rule Cross-Verification (Automated)

Every prediction is independently checked against the same monsoon thresholds used to build the training labels (Section 6.4). The backend recomputes the rule-based risk class from the actual rainfall readings and compares it to the model's class:

- **Agreement** → alert carries `ruleAgreed: true` and shows the "✓ verified" badge.
- **Mismatch** → logged with the exact evidence (rainfall, 3d, 7d) for audit; the alert is *never suppressed* — in a sensitive system, a warning stays visible even when verification fails.
- Verified in production: a live audit of 29 alerted districts recomputed all 29 rules from scratch and matched the stored verification exactly, including a genuine boundary-case mismatch (Dailekh: 150.4mm/7d crosses the 150mm high rule, while the model judged terrain/humidity and said moderate).

10.2 Ground-Truth Confirmation (Human)

Responders and admins can mark an alert **confirmed** or **not-confirmed** against what actually happened on the ground (with an optional note). The write is forward-only and recorded with the responder's identity and timestamp. Aggregated, this yields the *model confidence vs confirmed events* view: per risk band, how many alerts were confirmed, not-confirmed, or still pending — shown as a summary strip on the Alerts page and a dedicated card on the Reports page. This is the only place human truth enters the system, and it is kept visibly separate from model output.

11. Security & Reliability

Concern	Mitigation
Passwords	bcrypt (cost 12) hashing; password never returned by the API
Session/auth	JWT (7-day expiry), token-change invalidation via <code>passwordChangedAt</code>
Role-based access	<code>user / responder / admin</code> ; stats, resolve and confirm restricted to admin/responder
Sensor ingestion	JWT <i>or</i> shared <code>x-sensor-api-key</code> header
Headers	Helmet with a tuned Content-Security-Policy (allows the frontend's CDN libs and inline handlers)
Abuse	Rate limiting: global API limiter, stricter auth + sensor ingestion limiters
Log hygiene	Morgan log format redacts <code>?token=</code> so JWTs never reach log files
Payload size	JSON body capped at 10 kB
Resilience	ML service failure trips a 30s circuit-breaker then falls back to the local TF.js predictor; the free API budget is respected via concurrency + windows

12. Testing & Evaluation

12.1 Automated Tests

The backend ships a Jest + Supertest suite (smoke test of `GET /health` plus middleware-level coverage). Every change in this project was validated with:

- `node --check` syntax verification on every backend and frontend file.
- `npm test` (Jest) — passes.
- Python AST parse of the ML service before restart.

12.2 Live System Verification (Audit Performed)

Check	Result
Backend + ML service health	Both healthy; model v1.2.0, 77/77 districts
Data provenance	All live alerts/predictions/sensors sourced from the real weather feed — no mock/demo rows in production data
Ground-truth vs live weather	29/29 alerted districts: alert rainfall evidence matched a fresh Open-Meteo fetch to the decimal
Verification recomputation	29/29 rules recomputed independently matched the stored verification (including the 1 genuine mismatch, logged)
Fast-track polling	District bucket history showed the 30-min → 10-min window switch exactly when an alert became active
Frontend	All 5 pages render with real data, verification + confirmation badges visible, zero console errors

12.3 Head-to-Head Model Comparison

The current v1.2.0 models (trained on all 77 districts) were compared against a backup of the earlier 56-district run on live weather for 12 districts, scored against the documented rule. v1.2.0 agreed with the rule on 12/12 flood and 11/12 landslide cases; the older run agreed on only 1/12 and 2/12, systematically under-predicting risk. The comparison proved the value of completing the 21 remaining districts and confirmed the live models are the better ones.

13. Deployment & Operations

13.1 Services

Service	Command	Port
Backend	<code>node src/server.js</code> (from backend/)	5000
ML service	<code>python -m uvicorn predict_service:app --port 8000</code> (from models/)	8000
MongoDB	Local MongoDB instance	27017

13.2 Windows Auto-Start & Watchdog

- `start-services.ps1` — idempotent starter: only starts a service when its port is free, waits, and logs the truth of whether it came up (logs capped at 1 MB).
- `setup-scheduled-tasks.ps1` — registers a **2-minute watchdog** task (restarts any down service) and a **logon** task, plus a registry Run entry.
- Result: the system self-heals. Verified live — the watchdog task runs on schedule with result 0 and services recover automatically if killed.

13.3 Demo Credentials

Email: `admin@flds.demo`
Password: `demo12345` (role: admin – created by ``npm run seed:demo``)

14. Limitations & Future Work

14.1 Honest Limitations

- **Risk $\neq$ confirmed event.** The model predicts the likelihood that conditions meet documented danger thresholds; it does not observe actual flooding or slope failure. Only field reports confirm events (Section 10.2).
- **Label proxy.** Training labels come from the rainfall rule, not a confirmed-event database. The model inherits any imperfection of that rule.
- **Missing physical features.** No free live river-discharge or national slope dataset exists for Nepal, so `discharge` and `slope` are effectively constant in training. Soil moisture at training time is a rain-driven proxy (live inference uses real soil moisture).
- **Forecast vs archive gap.** Live inference uses forecast data while training used the archive — the distributions are close but not identical.
- **No ground-truth event history yet.** The confirmation feature is new; the confirmation-rate metric becomes statistically meaningful as more field reports accumulate.

14.2 Future Work

- **Add real river-gauge integration** (DHM) where available to replace the constant discharge feature with live data.
- **Geospatial hazard layers** — actual slope, land-use, and exposure data per district for richer features.
- **Confirmed-event backfill** — cross-train on historical disaster event records to move from rule-derived to event-derived labels.
- **Short-term rainfall nowcasts** (6–24 h forecasts) to turn risk scoring into true early *warning* lead time.
- **SMS/IVR mass-notification integration** and mobile push to expand alert reach beyond the dashboard.
- **Time-series models** (e.g., gradient boosting on lagged sequences) to capture multi-day wetness dynamics more explicitly.

15. Conclusion

Nepal FLDS demonstrates that a nationwide, sensor-free, near-real-time flood and landslide early warning system can be built entirely from free, publicly available weather data and open-source machine learning. Two XGBoost classifiers — trained on 241,318 district-days of real observed weather per hazard and achieving 99.8% / 99.7% accuracy — convert live Open-Meteo readings into per-district risk assessments for all 77 districts. A Node.js backend orchestrates the pipeline, persists full provenance in MongoDB, raises alerts, and streams them to a responsive web dashboard in real time over SSE.

The project's defining engineering choices are its **honesty mechanisms**: displayed probabilities are capped to risk bands so they can never contradict their labels; every alert is independently cross-checked against the documented monsoon thresholds; and human field confirmations are recorded separately, yielding an auditable "model confidence vs confirmed events" measure. Combined with automated service supervision, the system is a complete, runnable, and verifiable early-warning platform — and a strong demonstration of applying machine learning to a genuine national-scale disaster-risk problem.

Nepal FLDS — Final Year Project Report · XGBoost v1.2.0 · 77 Districts